

Descrierea Temei 1 - APD

Banescu Ema-Ioana, Grupa 331CB

Sectiunea 1:

Personal, tema mi s-a parut extrem de interesanta, ca si tematica, dar si datorita faptului ca a presupus implementarea in Java a unei aplicatii multi-threading, foarte utila in practica.

La inceput, cred ca partea de setup cu Maven, alcatuirea fisierului pom.xml, apoi schimbarea lui pentru a putea rula proiectul si pe checkerul de pe moodle, mi s-a parut putin dificil de facut, pentru ca a trebuit sa rezolv destul de multe erori, unele fisiere nu erau vazute de catre fisierul sursa create in main/java/tema/apd atunci cand se executa Makefile-ul. De asemenea, ceva care mie una mi s-a parut putin confuzing, a fost metoda de testare, adica la o rulare locala lucurile pareau corecte si scalabile, adica in parametrii, insa checkerul cateodata imi dadea maximul de puncte, dar alteori imi picau 1 sau 2 teste de scalabilitate, avand eroarea ca acceleratia medie ar fi prea mica, desi erau scalabile..

Ca si tematica, mi-a placut faptul ca a fost ceva practic, ceva care poate fi mai apoi dezvoltat, poate chiar integrat intr-o arhitectura mult mai complexa, cu parte de front-end, baza de date, si de asemenea mi-a placut ca a presupus interactiunea cu fisierul json in java, lucru care nu a fost atins in cadrul materiei Programare Orientata Obiect.

Sectiunea 2: Strategia de paralelizare:

2.1 Modul in care am impartit munca dintre threaduri

Abordarea a fost una clasica, am avut initial intentia de a folosi interfata ExecutorService, insa m-am gandit ca totusi o abordare simpla cu threaduri si cu impartirea pe etape pt fiecare thread fiind alocat un numar de operatiuni de la idx_start -> idx_end ar fi mai facila, intrucat in general ai mai mult control asupra prioritatilor, a granularitatii si a intregului ciclu de viata.

Avantajele thread polling-ul ar fi fost reutilizarea threadurilor (interzisa prin cerinta), bun pentru taskuri de scurta durata si cand am multe taskuri independente. Insa la mine taskurile depind de niste faze sincrone, am folosit bariere pentru acest tip de sincronizare si de divizare a computatiilor pe cateva etape.

Asa ca am mers pe varianta clasica in care fiecare thread proceseaza un interval static de date.

(prin formulele deduse si la lab: $start = id * N / P$ si $end = (id + 1) * N / P$. Deci pentru un thread,

a trebuit sa retin ca atribut (field) id-ul lui unic, ce portiune din lista de articole sa parcurga atunci cand voiam sa elimin duplicatele si ce structuri locale (preponderent hashMap-uri) sa populeze.

Etapa 1: Citirea fisierelor JSON.

La nivel de thread, fiecare preia numele urmatorului fisier de interes, prin "index = filesContor.getAndIncrement()". M-am gandit sa nu impart citirea fisierelor in mod static, pentru ca poate nu pe exemplele din cadrul testelor automate, dar in general as fi putut avea fisiere cu foarte putine articole, unele cu foarte multe articole.

Astfel, mi se pare ca abordarea mea distribuie totul mai echilibrat si gestioneaza mai bine situatiile in care un thread termina treaba mult prea devreme.

La finalul acestei etape, am toate articolele de tipul (Article) stocate intr-o lista Collections.synchronizedList

Etapa 2: - Partea de eliminare a duplicatelor

2.1

Dupa ce mi-am stocat articolele in lista si am asteptat la bariera, impart intervalul 0 - size_allArticlesList in range-uri pentru fiecare thread.

As fi putut foarte simplu sa folosesc, pentru identificarea elementelor duplicate, doua ConcurrentHashMap-uri globale (uuidFreq si titlesFreq) si sa fac actualizari concurente prin merge(). Aceasta chiar era abordarea mea initiala. Desi corecta (si cu scor mai mare pe local..), m-am gandit ca ar produce foarte mult lock contention pentru ca toate thread-urile se bat pe aceleasi chei.

Astfel ca, fiecare thread isi creeaza 2 hashMap-uri locale, localUuidFreq si localTitleFreq. Introduc o bariera, apoi dupa bariera fiecare thread va face merge in structurile globale.

Apoi iar astept la bariera, intrucat vreau ca toate sa fie introduse in Collection.synchronizedList-ul de articole unice, iar apoi pot trece la pasul 2.2

2.2

Acum, fiecare thread va prelucra doar articolele unice din propriul lor interval. Astfel ca pentru fiecare articol, am creat si am updatat map-urile locale, care memoreaza informatiile urmatoare:

localCatMap: maparea categoriei -> lista de uuids

localLangMap: maparea limbii -> lista de uuids

localCategoryCount: maparea categoriei -> numar de articole cu acea categorie

localLanguageCount: maparea limbii -> nr de articole unice scrise in acea limba.

Am folosit hashMap-uri normale, intrucat sunt accesate in mod exclusiv de catre threadul curent si nu ar fi existat vreun risc ca 2 sau mai multe thread-uri sa scrie in aceiasi structura.

Dupa finalizarea pasului de mai sus, am facut merge in structurile globale: mapCatIndices, mapLangIndices, mapCatCounts, mapLangCounts. (aici am folosit functia `merge`).

Per total, in defavoarea abordarii cu o singura zona globala partajata si atat (fara structuri locale), solutia de mai sus minimizeaza lock contention-ul, reduce iterarea prin structuri globale si creste scalabilitatea, lucru observat prin testare atat manuala cat si automata.

Etapa 3: - Identificarea celui mai recent articol

3.1 mostRecentArticle

Aici, thread-urile parcurg articolele unice din range-ul aferent fiecaruia, mentinand o variabila localRecentArticle (comparatii bazate pe published, apoi lexicografic, dupa uuid). Apoi, fiecare thread va actualiza variabila globala mostRecentArticle, dar am asigurat corectitudinea, introducand asta in blocul `synchronized` pe lockul Tema1.mostRecentArticleLock

3.2 Merge-uri

Mai devreme la o etapa anterioara, populasem structurile locale -> acum fiecare thread le va da merge in hashMap-urile globale.

4 Filtrarea cuvintelor de interes

Fiecare thread se va uita in range-ul sau, in uniqueArticles, va face prelucrarile pe text, din cerinta, aici am folosit si un Set local ca sa evit contorizarea in mod repetat a aceluasi cuvânt. Apoi am actualizat global hashMap-ul keyWordCount prin merge().

5 Generarea rezultatelor finale

Aici, threadul cu id-ul 0 (am ales eu sa il desemnez pe el), dupa ULTIMA bariera, acest thread va fi cel care se va ocupa de generarea raportului final (statisticilor). A se vedea snippeturile de cod aici (aici nu intru prea mult in detalii, intrucat nu am avut mari mecanisme de sincronizare, multe actualizari de maxime, dupa diverse criterii, am avut variabilele globale, deja computeate, si din acelea mi-am extras eu rezultatele dorite). Sunt practic doar look-up-uri prin structurile globale din Tema1 (exemplu: keyWordCount si extragerea cuvântului cu cel mai mare nr de articole in care il gases.)

Sectiunea 3: Analiza de performanta si scalabilitatea

I) Setup-ul de testare:

Configuratia sistemului

Testele au fost rulate pe urmatoarea configuratie hardware si software:

- CPU: AMD Ryzen 7 5800H
- Arhitectura: x86_64
- Numar nuclee fizice: 8
- Numar thread-uri: 16
- Threads per core: 2

Memorie RAM (WSL2):

- RAM total disponibil in WSL2: 7.4 GiB
- RAM liber: 6.5 GiB
- Swap: 2.0 GiB (neutilizat)

Sistem de operare:

- WSL2 Ubuntu

- Kernel Linux: 6.6.87.2-microsoft-standard-WSL2
- Host OS: Windows 10 Home

Versiunea de Java, pentru rularea programului:

- OpenJDK 11.0.29 (64-bit)
- Build: 11.0.29+7-post-Ubuntu-1ubuntu122.04

P	Run_1 (s)	Run_2 (s)	Run_3 (s)	Timp mediu
1	22.412	22.376	16.969	20.586
2	10.717	10.554	11.098	10.790
3	8.741	8.780	9.212	8.911
4	7.865	7.910	7.913	7.896
5	7.089	7.528	7.217	7.278
6	7.158	7.005	7.235	7.133
7	6.939	7.092	6.960	6.997
8	6.893	6.864	6.986	6.914
9	6.934	6.979	7.182	7.032
10	6.878	6.878	6.842	6.866
20	9.581	8.596	8.550	8.909

Snippeturi din cadrul terminalului, in care am vizualizat detaliile de mai sus, le atasez mai jos:

De asemenea, si comenzile cu timpii de rulare, precum si scriptul python in care am realizat graficele.

Comenzi in terminal in care am vizualizat specificatiile:

banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src\$ lscpu

Architecture: x86_64

CPU op-mode(s): 32-bit, 64-bit

Address sizes: 48 bits physical, 48 bits virtual

Byte Order: Little Endian

CPU(s): 16

On-line CPU(s) list: 0-15

Vendor ID: AuthenticAMD

Model name: AMD Ryzen 7 5800H with Radeon Graphics

CPU family: 25
Model: 80
Thread(s) per core: 2
Core(s) per socket: 8
Socket(s): 1
Stepping: 0

Sistemul de operare – Proiectul din IntelliJ l-am rulat cu ajutorul terminalului integrat in IDE, folosind wsl2

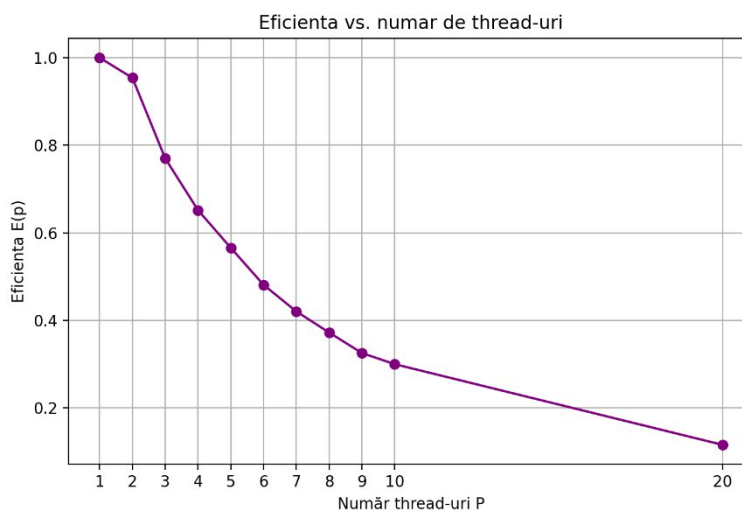
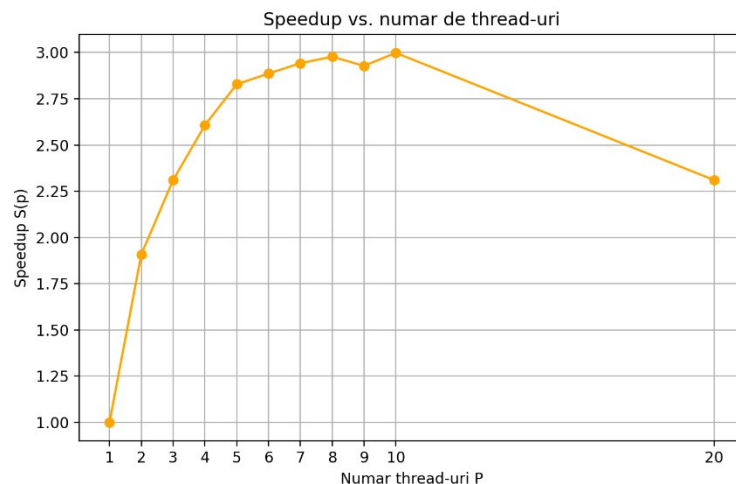
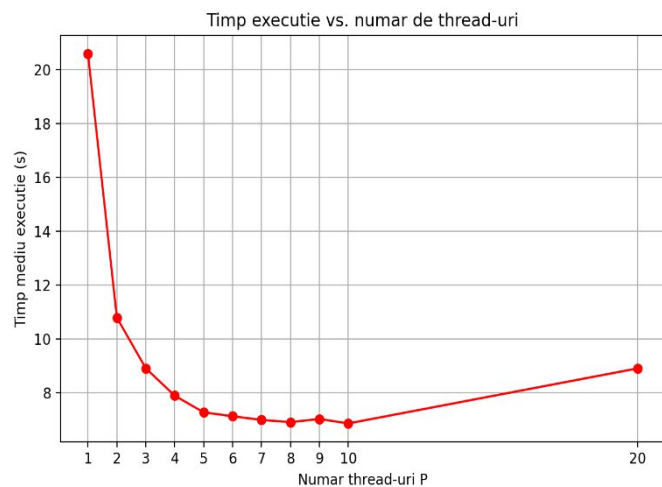
Linux DESKTOP-L7D15E1 6.6.87.2-microsoft-standard-WSL2 #1 SMP
PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux

Detalii despre memoria RAM disponibila in Linux

	total	used	free	shared	buff/cache	available
Mem:	7.4Gi	583Mi	6.5Gi	3.0Mi	355Mi	6.7Gi
Swap:	2.0Gi	0B	2.0Gi			

Versiunea de Java:

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ java -version
openjdk version "11.0.29" 2025-10-21
OpenJDK Runtime Environment (build 11.0.29+7-post-Ubuntu-1ubuntu122.04)
OpenJDK 64-Bit Server VM (build 11.0.29+7-post-Ubuntu-1ubuntu122.04, mixed mode,
sharing)
```



Dataset-ul folosit: cel din cadrul testului automat, test_5, care are 460 KB. L-am ales pe acesta, cu input.txt si articles.txt, intrucat era cel mai mare ca si dimensiune, deci am putut trage o concluzie relativ pertinenta in urma masuratorilor, cat mai apropiata de realitate.

Interpretare

Am utilizat comanda time, „time make run”

Unde m-a interesat timpul real (adica timpul de pe ceas), cat a durat efectiv comanda data, din perspectiva mea, a programatorului. Comanda time mi-a mai oferit detalii legate de user (suma timpilor de pe CPU pe care ii petrecem in user mode, adica toate threadurile si toate core-urile), si sys, timpul pe care il petrece in kernel cu apeluri de sistem etc.

Am adunat toate datele (de la categoria „real” si le am colectat intr-o lista de timpi in scriptul python de mai jos, apoi am facut media fiecarui triplet de timpi, pentru fiecare thread (din cele 3 rulari succesive), si apoi doar am aplicat formulele pt speedup si eficienta

Analiza rezultatelor:

Pentru 1 THREAD: real-ul a fost aprox. 20 de secunde.

Pentru 2 threaduri, real-ul a devenit 10.8 s, aprox. Deci cam de 2 ori mai rapid. User-ul am observat ca a crescut, tocmai pentru ca am 2 core-uri ocupate putin mai mult timp in paralel.

Apoi cu 3, 4, 5, 6, 7, 8, real time-ul scade pana la aprox. 7 secunde.

Am observat ca atata timp cat cresc numarul de thread-uri pana aproape la nr meu de core-uri, este vizibil un speedup real, care in mod evident, NU este unul liniar.

Evident ca exista o limita de paralelism hardware, adica numarul de core-uri fizice de pe procesor, dar intervine si legea lui Amdahl, ca mereu exista o bucatica de cod secventiala care NU se poate paraleliza. Astfel, ca atunci cand cresc foarte mult numarul de threaduri din program, timpul la real ramane cam constant, se plafoneaza la aprx 7 secunde. Atunci cand am numeroase thread-uri, timpul de executie este limitat de overhead de sincronizare, de scheduling, iar timpul user va fi extrem de mare, intrucat va aparea context switching extrem de marit, lock-uri interne, barierele pe care eu le-am definit influenteaza si ele, si threadurile risca sa stea mult in asteptare.

Astfel ca valoarea maxima a speedup-ului este atinsa in zona de 6-8 threaduri, iar cresterea nr de threaduri peste, il va reduce, intrucat apar efecte nedorite, inclusiv coliziuni pe ConcurrentHashMap.

Scriptul unde am calculat speedup-ul si eficienta, si am reprezentat si grafic:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# timpii mei masurati cu time
```

```
real_time = {
```

```

1: [22.412, 22.376, 16.969],
2: [10.717, 10.554, 11.098],
3: [8.741, 8.780, 9.212],
4: [7.865, 7.910, 7.913],
5: [7.089, 7.528, 7.217],
6: [7.158, 7.005, 7.235],
7: [6.939, 7.092, 6.960],
8: [6.893, 6.864, 6.986],
9: [6.934, 6.979, 7.182],
10: [6.878, 6.878, 6.842],
20: [9.581, 8.596, 8.550]
}

# Procesarea timpilor:
# 1. calculul mediei timpilor celor 3 rulari, apoi speedup-ul si eficienta.

averageRealTime = {}

for p in real_time:
    media = np.mean(real_time[p])
    averageRealTime[p] = media

# cat este T(1) deci media celor 3 rulari pt varianta secventiala, cu doar 1 thread.
T1 = averageRealTime[1]

# speedupul: T(1) / T(p)
speedup = {}
for p in averageRealTime:
    speedup[p] = T1 / averageRealTime[p]

# Eficienta:
efficiency = {}
for p in speedup:
    efficiency[p] = speedup[p] / p

# imi construiesc listele, in ordinea crescatoare a nr de threaduri, p.
P_axis = sorted(averageRealTime.keys())
avg_list = []
speedup_list = []
eff_list = []

for p in P_axis:
    avg_list.append(averageRealTime[p])
    speedup_list.append(speedup[p])
    eff_list.append(efficiency[p])

# Graficele cu matplotlib.

plt.figure(figsize=(8,5))
plt.plot(P_axis, avg_list, marker='o', color='red')
plt.xlabel("Numar thread-uri P")
plt.ylabel("Timp mediu executie (s)")
plt.title("Timp executie vs. numar de thread-uri")
plt.grid(True)
plt.xticks(P_axis)
plt.savefig("curba_timp.png", dpi=200)

# graficul pentru speedup:

plt.figure(figsize=(8,5))
plt.plot(P_axis, speedup_list, marker='o', color='orange')
plt.xlabel("Numar thread-uri P")

```

```
plt.ylabel("Speedup S(p)")
plt.title("Speedup vs. numar de thread-uri")
plt.grid(True)
plt.xticks(P_axis)
plt.savefig("curba_speedup.png", dpi=200)
```

graficul eficientei:

```
plt.figure(figsize=(8,5))
plt.plot(P_axis, eff_list, marker='o', color='purple')
plt.xlabel("Numar thread-uri P")
plt.ylabel("Eficienta E(p)")
plt.title("Eficienta vs. numar de thread-uri")
plt.grid(True)
plt.xticks(P_axis)
plt.savefig("curba_eficienta.png", dpi=200)
```

Timpii de rulare, din terminal:

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="1 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m22.412s
```

```
user 0m29.076s
```

```
sys 0m3.399s
```

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="1 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m22.376s
```

```
user 0m29.471s
```

```
sys 0m3.588s
```

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="1 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m16.969s
```

```
user 0m35.232s
```

```
sys 0m3.258s
```

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="2 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m10.717s
```

```
user 0m36.863s
```

```
sys 0m3.120s
```

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="2 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m10.554s
```

user 0m36.790s

sys 0m3.026s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="2 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m11.098s

user 0m37.891s

sys 0m3.988s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="3 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m8.741s

user 0m36.640s

sys 0m3.783s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="3 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m8.780s

user 0m38.163s

sys 0m3.121s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="3 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m9.212s

user 0m37.850s

sys 0m4.591s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="4 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.865s

user 0m39.046s

sys 0m3.810s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="4 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.910s

user 0m39.703s

sys 0m4.432s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="4 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.913s

user 0m41.074s

sys 0m4.018s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="5 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.089s

user 0m40.435s

sys 0m3.375s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="5 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.528s

user 0m43.209s

sys 0m3.697s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="5 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.217s

user 0m36.418s

sys 0m4.482s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="6 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.158s

user 0m44.888s

sys 0m4.179s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="6 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.005s

user 0m44.409s

sys 0m3.458s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="6 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.235s

user 0m46.172s

sys 0m3.364s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="7 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m6.939s

user 0m47.169s

sys 0m4.487s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="7 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.092s

user 0m42.131s

sys 0m4.476s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run  
ARGS="7 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m6.960s

user 0m49.160s

sys 0m3.393s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run  
ARGS="8 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m6.893s

user 0m50.037s

sys 0m4.448s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run  
ARGS="8 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m6.864s

user 0m52.879s

sys 0m3.951s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run  
ARGS="8 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m6.986s

user 0m52.994s

sys 0m4.761s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run  
ARGS="9 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m6.934s

user 0m54.519s

sys 0m5.142s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run  
ARGS="9 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m6.979s

user 0m55.733s

sys 0m4.071s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run  
ARGS="9 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

real 0m7.182s

user 0m56.082s

sys 0m5.635s

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run  
ARGS="10 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m6.878s
user 0m53.299s
sys 0m5.263s
```

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="10 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m6.878s
user 0m59.152s
sys 0m3.769s
```

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="10 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m6.842s
user 0m56.849s
sys 0m3.711s
```

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="20 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m9.581s
user 1m13.939s
sys 0m8.159s
```

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="20 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m8.596s
user 1m15.120s
sys 0m7.667s
```

```
banescuema@DESKTOP-L7D15E1:~/TEMA_APD_1/tema_apd_1/src$ time make run
ARGS="20 ../checker/input/tests/test_5/articles.txt ../checker/input/tests/test_5/inputs.txt" > /dev/null
```

```
real 0m8.550s
user 1m16.542s
sys 0m5.493s
```